

Функции в C++: подробное объяснение

1. Что такое функция и зачем она нужна?

Функция - это именованный блок кода, который выполняет определенную задачу и может быть вызван из других частей программы.

Зачем нужны функции:

- Повторное использование кода (не копируем один и тот же код)
- Упрощение программы (разбиваем на логические части)
- Улучшение читаемости
- Облегчение отладки и тестирования

2. Объявление и определение функции

Объявление функции (прототип)

```
// Объявление (прототип) функции
// Сообщает компилятору о существовании функции
int sum(int a, int b); // точка с запятой в конце!
```

Определение функции

```
// Определение функции
// Содержит фактическую реализацию
int sum(int a, int b) { // фигурные скобки
    return a + b;
}
```

Полный пример

```
#include <iostream>
using namespace std;

// 1. Объявление функций (прототипы)
int sum(int a, int b);
void printMessage();
```

```
int main() {
    // Вызов функции
    int result = sum(5, 3);
    cout << "Сумма: " << result << endl;

    printMessage();

    return 0;
}

// 2. Определение функции sum
int sum(int a, int b) {
    return a + b; // возвращаем результат
}

// 3. Определение функции printMessage
void printMessage() {
    cout << "Привет из функции!" << endl;
    // Функция void ничего не возвращает
}
```

3. Параметры по значению (Value Parameters)

Параметр по значению создает КОПИЮ передаваемого значения. Изменения параметра внутри функции НЕ влияют на оригинальную переменную.

```
#include <iostream>
using namespace std;

// Параметры по значению
void incrementByValue(int x) {
    x = x + 10; // изменяем КОПИЮ
    cout << "Внутри функции: x = " << x << endl;
}

int main() {
    int number = 5;

    cout << "До вызова функции: number = " << number << endl;

    incrementByValue(number); // передаем КОПИЮ значения

    cout << "После вызова функции: number = " << number << endl;
    // number остался равен 5, потому что функция работала с копией

    return 0;
}
```

```
/*  
Вывод:  
До вызова функции: number = 5  
Внутри функции: x = 15  
После вызова функции: number = 5  
*/
```

4. Параметры по ссылке (Reference Parameters)

Параметр по ссылке работает с **ОРИГИНАЛЬНОЙ** переменной. Изменения параметра внутри функции влияют на оригинальную переменную.

```
#include <iostream>  
using namespace std;  
  
// Параметр по ссылке (знак &)  
void incrementByReference(int &x) {  
    x = x + 10; // изменяем ОРИГИНАЛ  
    cout << "Внутри функции: x = " << x << endl;  
}  
  
// Пример с обменом значений  
void swap(int &a, int &b) {  
    int temp = a;  
    a = b;  
    b = temp;  
}  
  
int main() {  
    int number = 5;  
  
    cout << "До вызова функции: number = " << number << endl;  
  
    incrementByReference(number); // передаем ССЫЛКУ на переменную  
  
    cout << "После вызова функции: number = " << number << endl;  
    // number изменился на 15!  
  
    // Пример с обменом значений  
    int x = 10, y = 20;  
    cout << "\nДо swap: x = " << x << ", y = " << y << endl;  
  
    swap(x, y); // передаем ссылки на x и y  
  
    cout << "После swap: x = " << x << ", y = " << y << endl;
```

```

    return 0;
}

/*
Вывод:
До вызова функции: number = 5
Внутри функции: x = 15
После вызова функции: number = 15

До swap: x = 10, y = 20
После swap: x = 20, y = 10
*/

```

5. Константные параметры (Const Parameters)

Константные параметры гарантируют, что функция не изменит переданное значение.

```

#include <iostream>
using namespace std;

// Константный параметр по ссылке
// Эффективно (не создает копию) и безопасно (не изменяет оригинал)
void printArray(const int arr[], int size) {
    // arr[] - передача массива по ссылке (фактически по указателю)
    // const гарантирует, что массив не будет изменен

    cout << "Массив: ";
    for(int i = 0; i < size; i++) {
        cout << arr[i] << " ";
        // arr[i] = 0; // ОШИБКА! Нельзя изменить константный массив
    }
    cout << endl;
}

// Константная ссылка на int
void printValue(const int &value) {
    cout << "Значение: " << value << endl;
    // value = 100; // ОШИБКА! Нельзя изменить константную ссылку
}

int main() {
    int numbers[5] = {1, 2, 3, 4, 5};
    int x = 42;

    printArray(numbers, 5);
    printValue(x);
}

```

```
    return 0;
}
```

6. Возвращаемые значения

Функции могут возвращать значения или не возвращать ничего (void).

```
#include <iostream>
using namespace std;

// Возвращает int
int add(int a, int b) {
    return a + b;
}

// Возвращает double
double average(int a, int b) {
    return (a + b) / 2.0; // 2.0 для вещественного деления
}

// Возвращает bool
bool isEven(int number) {
    return (number % 2 == 0);
}

// Возвращает указатель на массив (продвинутая тема)
int* createArray(int size) {
    int* arr = new int[size]; // выделяем память
    for(int i = 0; i < size; i++) {
        arr[i] = i * i; // заполняем квадратами чисел
    }
    return arr; // возвращаем указатель
}

// Ничего не возвращает
void printStars(int count) {
    for(int i = 0; i < count; i++) {
        cout << "* ";
    }
    cout << endl;
}

int main() {
    int sum = add(10, 20);
    cout << "Сумма: " << sum << endl;

    double avg = average(10, 20);
```

```

cout << "Среднее: " << avg << endl;

cout << "Число 5 четное? " << (isEven(5) ? "Да" : "Нет") << endl;

printStats(5);

// Работа с возвращаемым указателем
int* myArray = createArray(5);
cout << "Созданный массив: ";
for(int i = 0; i < 5; i++) {
    cout << myArray[i] << " ";
}
cout << endl;

delete[] myArray; // освобождаем память!

return 0;
}

```

7. Рекурсия

Рекурсия - когда функция вызывает саму себя. Должна иметь условие выхода!

Пример 1: Факториал

```

#include <iostream>
using namespace std;

// Рекурсивная функция для вычисления факториала
// n! = n * (n-1) * (n-2) * ... * 1
// 0! = 1
int factorial(int n) {
    // Базовый случай (условие выхода из рекурсии)
    if(n <= 1) {
        return 1;
    }
    // Рекурсивный случай
    return n * factorial(n - 1);
}

int main() {
    int n = 5;
    cout << "Факториал " << n << " = " << factorial(n) << endl;

    // Как это работает для factorial(5):
    // factorial(5) = 5 * factorial(4)
    // factorial(4) = 4 * factorial(3)

```

```
// factorial(3) = 3 * factorial(2)
// factorial(2) = 2 * factorial(1)
// factorial(1) = 1 ← базовый случай!
// Затем все возвращается назад:
// factorial(2) = 2 * 1 = 2
// factorial(3) = 3 * 2 = 6
// factorial(4) = 4 * 6 = 24
// factorial(5) = 5 * 24 = 120

return 0;
}
```

Пример 2: Числа Фибоначчи

```
#include <iostream>
using namespace std;

// Рекурсивная функция для чисел Фибоначчи
// F(0) = 0, F(1) = 1
// F(n) = F(n-1) + F(n-2)
int fibonacci(int n) {
    // Базовые случаи
    if(n == 0) return 0;
    if(n == 1) return 1;

    // Рекурсивный случай
    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    cout << "Числа Фибоначчи: ";
    for(int i = 0; i < 10; i++) {
        cout << fibonacci(i) << " ";
    }
    cout << endl;

    return 0;
}
```

Пример 3: Рекурсивный поиск в массиве

```
#include <iostream>
using namespace std;

// Рекурсивный поиск максимума в массиве
int findMax(int arr[], int start, int end) {
```

```

// Базовый случай: один элемент
if(start == end) {
    return arr[start];
}

// Рекурсивный случай
int mid = (start + end) / 2;
int max1 = findMax(arr, start, mid);    // максимум в левой половине
int max2 = findMax(arr, mid + 1, end);  // максимум в правой половине

// Возвращаем больший из двух максимумов
return (max1 > max2) ? max1 : max2;
}

int main() {
    int arr[] = {3, 7, 2, 9, 1, 4, 6, 8, 5};
    int size = 9;

    int max = findMax(arr, 0, size - 1);
    cout << "Максимальный элемент: " << max << endl;

    return 0;
}

```

8. Параметры по умолчанию

```

#include <iostream>
using namespace std;

// Функция с параметрами по умолчанию
void printInfo(string name, int age = 18, string city = "Москва") {
    cout << "Имя: " << name << endl;
    cout << "Возраст: " << age << endl;
    cout << "Город: " << city << endl;
    cout << "---" << endl;
}

int main() {
    // Можно вызывать функцию с разным количеством параметров
    printInfo("Анна", 25, "Санкт-Петербург");
    printInfo("Иван", 30);                // city будет "Москва" по умолчанию
    printInfo("Мария");                  // age=18, city="Москва" по умолчанию

    // Важно: параметры по умолчанию должны идти в конце списка
    // void printInfo(string name = "Гость", int age, string city) - ОШИБКА
}

```



```
    return 0;  
}
```

9. Перегрузка функций

```
#include <iostream>  
using namespace std;  
  
// Перегрузка функций - несколько функций с одним именем,  
// но разными параметрами  
  
// 1. Сложение двух целых чисел  
int add(int a, int b) {  
    return a + b;  
}  
  
// 2. Сложение трех целых чисел  
int add(int a, int b, int c) {  
    return a + b + c;  
}  
  
// 3. Сложение двух вещественных чисел  
double add(double a, double b) {  
    return a + b;  
}  
  
// 4. Конкатенация строк  
string add(string a, string b) {  
    return a + b;  
}  
  
int main() {  
    cout << "add(5, 3) = " << add(5, 3) << endl;          // вызов 1-й фу  
    cout << "add(5, 3, 2) = " << add(5, 3, 2) << endl;      // вызов 2-й фу  
    cout << "add(3.14, 2.71) = " << add(3.14, 2.71) << endl; // вызов 3-й  
    cout << "add(\"Hello, \", \"World!\") = "  
        << add(string("Hello, "), string("World!")) << endl; // вызов 4-  
  
    return 0;  
}
```



Практические задания

Задание 1: Калькулятор

Создайте функции для всех арифметических операций (+, -, *, /).

Задание 2: Работа с массивами

Создайте функции для:

- Поиска минимума/максимума
- Вычисления среднего значения
- Реверсирования массива

Задание 3: Рекурсивные задачи

1. Вычисление суммы цифр числа
2. Проверка палиндрома (слово, которое читается одинаково слева направо)
3. Быстрое возведение в степень

Задание 4: Функции с разными типами параметров

Создайте функцию, которая принимает параметры по значению, по ссылке и по константной ссылке, и продемонстрируйте разницу.

Важные правила и советы

1. **Одна функция - одна задача**
2. **Именуйте функции глаголами** (calculate, print, find, etc.)
3. **Избегайте глобальных переменных** - передавайте данные через параметры
4. **Рекурсия должна иметь условие выхода!**
5. **Используйте const** для параметров, которые не должны изменяться
6. **Для больших объектов** (структуры, классы, массивы) передавайте по ссылке
7. **Для простых типов** (int, double, char) можно передавать по значению

Типичные ошибки

```
// 1. Бесконечная рекурсия (нет условия выхода)
void infiniteRecursion(int x) {
    infiniteRecursion(x + 1); // ОШИБКА! Никогда не закончится
}
```

```
// 2. Изменение константного параметра
void print(const int &x) {
    x = 10; // ОШИБКА! x - константная ссылка
```

```

}

// 3. Возврат ссылки на локальную переменную
int& badFunction() {
    int x = 10;
    return x; // ОШИБКА! x уничтожится после выхода из функции
}

```

Функции - это строительные блоки программ на C++. Освоив их, вы сможете создавать хорошо структурированные, читаемые и поддерживаемые программы!

Все виды синтаксиса параметров функций в C++

Полный обзор синтаксиса параметров

1. Параметры по значению (Pass by Value)

Базовый синтаксис:

```

// Простой тип
void function(int parameter) { /* ... */ }

// Несколько параметров
void function(int a, double b, char c) { /* ... */ }

// Указатель как значение (копируется указатель, не данные)
void function(int* ptr) { /* ... */ }

```

Полный пример:

```

#include <iostream>

// 1. Простые типы по значению
void simpleByValue(int num, double price, char grade) {
    // Все параметры - локальные копии
    num = 100;           // Меняется только копия
    price = 99.99;       // Меняется только копия
    grade = 'A';         // Меняется только копия

    std::cout << "Внутри функции: " << num << ", " << price << ", " << gr

```

```

}

// 2. Указатель по значению
void pointerByValue(int* ptr) {
    // ptr - копия указателя, указывает на те же данные
    *ptr = 100;          // Меняем данные по адресу
    ptr = nullptr;       // Меняем только копию указателя
}

// 3. Массив по значению (на самом деле передается как указатель!)
void arrayByValue(int arr[5]) { // Размер игнорируется!
    // arr на самом деле int* - указатель на первый элемент
    arr[0] = 100;          // Меняет оригинальный массив!
}

// 4. Структура по значению (полное копирование)
struct Point {
    int x, y;
};

void structByValue(Point p) {
    p.x = 100;             // Меняется только копия
    p.y = 200;
}

int main() {
    int a = 5;
    double b = 10.5;
    char c = 'B';

    std::cout << "До вызова: " << a << ", " << b << ", " << c << std::endl;
    simpleByValue(a, b, c);
    std::cout << "После вызова: " << a << ", " << b << ", " << c << std::endl;

    // Пример с указателем
    int value = 50;
    int* ptr = &value;
    pointerByValue(ptr);
    std::cout << "value после pointerByValue: " << value << std::endl;

    // Пример с массивом
    int arr[5] = {1, 2, 3, 4, 5};
    arrayByValue(arr);
    std::cout << "arr[0] после arrayByValue: " << arr[0] << std::endl;

    // Пример со структурой
    Point point = {10, 20};
    structByValue(point);
    std::cout << "point после structByValue: (" << point.x << ", " << poi

```

```
    return 0;
}
```

2. Параметры по ссылке (Pass by Reference)

Базовый синтаксис:

```
// Ссылка на простой тип
void function(int& ref) { /* ... */ }
```

```
// Ссылка на константу
void function(const int& ref) { /* ... */ }
```

```
// Ссылка на указатель
void function(int*& refPtr) { /* ... */ }
```

```
// Ссылка на массив с размером
void function(int (&refArr)[5]) { /* ... */ }
```

Полный пример со всеми видами ссылок:

```
#include <iostream>
#include <array>

// 1. Обычная ссылка (можем изменять)
void modifyByReference(int& ref) {
    ref *= 2; // Меняем оригинальное значение
}

// 2. Константная ссылка (только для чтения)
void readByConstReference(const int& cref) {
    // cref *= 2; // ОШИБКА: нельзя изменять константную ссылку
    std::cout << "Значение: " << cref << std::endl;
}

// 3. Ссылка на указатель
void modifyPointer(int*& ptrRef) {
    static int newValue = 100;
    ptrRef = &newValue; // Меняем сам указатель
}

// 4. Ссылка на константный указатель
void readPointer(const int* const& constPtrRef) {
    // constPtrRef - константная ссылка на константный указатель
    // Нельзя изменить ни указатель, ни данные
    std::cout << "Значение по указателю: " << *constPtrRef << std::endl;
}
```

```
}

// 5. Ссылка на массив фиксированного размера
void processArray(int (&arrRef)[5]) { // Должен совпадать размер!
    for(int i = 0; i < 5; i++) {
        arrRef[i] *= 2; // Меняем оригинальный массив
    }
}

// 6. Ссылка на многомерный массив
void processMatrix(int (&matrixRef)[3][3]) {
    for(int i = 0; i < 3; i++) {
        for(int j = 0; j < 3; j++) {
            matrixRef[i][j] += 1;
        }
    }
}

// 7. Ссылка на структуру
struct Data {
    int id;
    std::string name;
};

void modifyStruct(Data& dataRef) {
    dataRef.id = 100;
    dataRef.name = "Modified";
}

// 8. Константная ссылка на структуру
void printStruct(const Data& dataRef) {
    std::cout << "ID: " << dataRef.id << ", Name: " << dataRef.name << st
}

int main() {
    // 1. Обычная ссылка
    int x = 5;
    modifyByReference(x);
    std::cout << "x после modifyByReference: " << x << std::endl;

    // 2. Константная ссылка
    readByConstReference(x);

    // 3. Ссылка на указатель
    int* ptr = &x;
    modifyPointer(ptr);
    std::cout << "*ptr после modifyPointer: " << *ptr << std::endl;

    // 4. Константная ссылка на константный указатель
    const int* constPtr = &x;
```

```

readPointer(constPtr);

// 5. Ссылка на массив
int arr[5] = {1, 2, 3, 4, 5};
processArray(arr);
std::cout << "arr после processArray: ";
for(int val : arr) std::cout << val << " ";
std::cout << std::endl;

// 6. Ссылка на многомерный массив
int matrix[3][3] = {{1,2,3}, {4,5,6}, {7,8,9}};
processMatrix(matrix);
std::cout << "matrix[1][1] после processMatrix: " << matrix[1][1] <<

// 7-8. Ссылки на структуры
Data data = {1, "Original"};
modifyStruct(data);
printStruct(data);

return 0;
}

```

3. Комбинированные варианты

Смешанные параметры:

```

#include <iostream>

// Комбинация разных типов параметров
void mixedParameters(
    int byValue,           // По значению
    int& byRef,            // По ссылке
    const int& byConstRef, // По константной ссылке
    int* byPtrValue,       // Указатель по значению
    int*& byPtrRef,        // Ссылка на указатель
    const int* const& constPtrConstRef // Константная ссылка на констант
) {
    // Можем изменить: byRef, *byPtrValue, byPtrRef
    // Не можем изменить: byValue, byConstRef, constPtrConstRef

    byValue = 0;           // Меняем копию
    byRef = 1;             // Меняем оригинал
    // byConstRef = 2;     // Ошибка

    *byPtrValue = 3;       // Меняем данные по указателю
    byPtrValue = nullptr;  // Меняем копию указателя

    *byPtrRef = 4;         // Меняем данные по указателю
}

```

```

    byPtrRef = nullptr;    // Меняем сам указатель

    // *constPtrConstRef = 5;  // Ошибка
    // constPtrConstRef = nullptr;  // Ошибка
}

int main() {
    int a = 10, b = 20, c = 30;
    int* ptr1 = &b;
    int* ptr2 = &c;
    const int* const constPtr = &a;

    mixedParameters(a, b, c, ptr1, ptr2, constPtr);

    std::cout << "a = " << a << ", b = " << b << ", c = " << c << std::endl;
    std::cout << "*ptr1 = " << (ptr1 ? *ptr1 : 0) << std::endl;
    std::cout << "*ptr2 = " << (ptr2 ? *ptr2 : 0) << std::endl;

    return 0;
}

```

4. Специальные синтаксисы C++11 и новее

Rvalue-ссылки (C++11):

```

#include <iostream>
#include <utility>

// Rvalue-ссылка (только для временных объектов)
void processRvalue(int&& rref) {
    std::cout << "Обработка rvalue: " << rref << std::endl;
}

// Универсальная ссылка (шаблон)
template<typename T>
void universalReference(T&& uref) {
    std::cout << "Универсальная ссылка: " << uref << std::endl;
}

// Perfect forwarding (идеальная передача)
template<typename T>
void wrapper(T&& arg) {
    // std::forward сохраняет категорию значения (lvalue/rvalue)
    processRvalue(std::forward<T>(arg));
}

int main() {
    int x = 42;
}

```



```

// processRvalue(x);          // Ошибка: x - lvalue
processRvalue(100);          // OK: 100 - rvalue
processRvalue(std::move(x)); // OK: x превращен в rvalue

universalReference(x);       // OK: выводится как int&
universalReference(200);     // OK: выводится как int&&

wrapper(300);                // Perfect forwarding

return 0;
}

```

Параметры по умолчанию:

```

#include <iostream>

// Параметры по умолчанию
void defaultParameters(
    int required,
    int optional1 = 10,          // Значение по умолчанию
    double optional2 = 3.14,
    const char* optional3 = "default"
) {
    std::cout << required << ", " << optional1 << ", "
               << optional2 << ", " << optional3 << std::endl;
}

// Параметры по умолчанию только в конце!
// void badExample(int a = 1, int b) {} // ОШИБКА

int main() {
    defaultParameters(1);          // 1, 10, 3.14, "default"
    defaultParameters(1, 20);      // 1, 20, 3.14, "default"
    defaultParameters(1, 20, 2.71); // 1, 20, 2.71, "default"
    defaultParameters(1, 20, 2.71, "custom"); // 1, 20, 2.71, "custom"

    return 0;
}

```

Variadic templates (переменное число параметров):

```

#include <iostream>

// Базовый случай рекурсии
void printAll() {
    std::cout << std::endl;
}

```

```

}

// Шаблон с переменным числом параметров
template<typename T, typename... Args>
void printAll(T first, Args... args) {
    std::cout << first << " ";
    printAll(args...); // Рекурсивный вызов
}

// Fold expressions (C++17)
template<typename... Args>
void printAllFold(Args... args) {
    (std::cout << ... << args) << std::endl; // Унарная свертка
}

int main() {
    printAll(1, 2.5, "hello", 'A');           // 1 2.5 hello A
    printAllFold(1, 2.5, "hello", 'A');       // 12.5helloA

    return 0;
}

```

5. Указатели vs Ссылки: детальное сравнение

```

#include <iostream>

// 1. Указатель по значению
void pointerExample(int* ptr) {
    if (ptr) { // Всегда проверяем на nullptr!
        *ptr = 100; // Изменяем данные
        ptr = nullptr; // Меняем только локальную копию
    }
}

// 2. Указатель по ссылке
void pointerReference(int*& ptrRef) {
    if (ptrRef) {
        *ptrRef = 200; // Изменяем данные
        ptrRef = nullptr; // Меняем сам указатель
    }
}

// 3. Ссылка (никогда не может быть nullptr)
void referenceExample(int& ref) {
    ref = 300; // Всегда валидный объект
    // Не нужно проверять на null!
}

```

```
// 4. Константный указатель на константу
void constPointerConst(const int* const constPtr) {
    // Нельзя изменить ни указатель, ни данные
    // if (constPtr) std::cout << *constPtr << std::endl;
}

int main() {
    int value = 50;
    int* ptr = &value;

    std::cout << "Исходное значение: " << value << std::endl;

    pointerExample(ptr);
    std::cout << "После pointerExample: " << value << std::endl;
    std::cout << "ptr равен nullptr? " << (ptr == nullptr ? "да" : "нет")

    pointerReference(ptr);
    std::cout << "После pointerReference: " << value << std::endl;
    std::cout << "ptr равен nullptr? " << (ptr == nullptr ? "да" : "нет")

    ptr = &value; // Восстанавливаем указатель
    referenceExample(*ptr); // Разыменовываем для получения ссылки
    std::cout << "После referenceExample: " << value << std::endl;

    return 0;
}
```

6. Массивы как параметры: все варианты

```
#include <iostream>
#include <array>
#include <vector>

// 1. Массив как указатель (сишный стиль)
void arrayAsPointer(int* arr, size_t size) {
    for(size_t i = 0; i < size; i++) {
        arr[i] *= 2;
    }
}

// 2. Массив с указанием размера (размер игнорируется!)
void arrayWithSize(int arr[10]) { // 10 игнорируется!
    // Фактически это int* arr
}

// 3. Ссылка на массив фиксированного размера
void arrayReference(int (&arr)[5]) { // Размер важен!
    for(int& elem : arr) {
```

```
        elem += 1;
    }
}

// 4. Массив как шаблонный параметр (C++)
template<size_t N>
void templateArray(int (&arr)[N]) { // Выводит размер автоматически
    std::cout << "Размер массива: " << N << std::endl;
    for(size_t i = 0; i < N; i++) {
        arr[i] = i * 10;
    }
}

// 5. std::array (рекомендуется в C++)
void stdArrayExample(std::array<int, 5>& arr) {
    for(auto& elem : arr) {
        elem += 100;
    }
}

// 6. std::vector (динамический массив)
void vectorExample(std::vector<int>& vec) {
    for(auto& elem : vec) {
        elem += 1000;
    }
}

int main() {
    // 1. Сишный массив
    int cArray[5] = {1, 2, 3, 4, 5};
    arrayAsPointer(cArray, 5);

    // 3. Ссылка на массив
    int fixedArray[5] = {10, 20, 30, 40, 50};
    arrayReference(fixedArray);

    // 4. Шаблонный массив
    int dynamicSizeArray[] = {1, 2, 3}; // Размер 3
    templateArray(dynamicSizeArray);

    // 5. std::array
    std::array<int, 5> stdArr = {100, 200, 300, 400, 500};
    stdArrayExample(stdArr);

    // 6. std::vector
    std::vector<int> vec = {1000, 2000, 3000};
    vectorExample(vec);
}
```

```
    return 0;
}
```

7. Сравнительная таблица всех видов параметров

Тип параметра	Синтаксис	Модификация оригинала	Может быть null	Передача массива	C++ версия
По значению	<code>void f(T param)</code>	✗ Нет	✗ Нет	Только указатель	C++98
По ссылке	<code>void f(T& param)</code>	✓ Да	✗ Нет	<code>T (&)[N]</code>	C++98
По константной ссылке	<code>void f(const T& param)</code>	✗ Нет	✗ Нет	<code>const T (&)[N]</code>	C++98
Указатель по значению	<code>void f(T* param)</code>	*Да (через *)	✓ Да	Естественно	C++98
Ссылка на указатель	<code>void f(T*& param)</code>	✓ Да (и указатель, и данные)	✓ Да	-	C++98
Rvalue-ссылка	<code>void f(T&& param)</code>	✓ Временные объекты	✗ Нет	-	C++11
Универсальная ссылка	<code>template f(T&&)</code>	Зависит от типа	✗ Нет	-	C++11
<code>std::array</code>	<code>void f(array<T, N>&)</code>	✓ Да	✗ Нет	Идеально для массивов	C++11
<code>std::vector</code>	<code>void f(vector<T>&)</code>	✓ Да	✓ Пустой вектор	Динамические массивы	C++98

8. Лучшие практики и рекомендации

```
#include <iostream>
#include <vector>
#include <string>
```

// ПРАВИЛЬНО:

// 1. Простые типы - по значению

```
void processInt(int value) { /* ... */ }
```

// 2. Большие объекты - по константной ссылке

```
void printString(const std::string& str) { /* ... */ }
```

// 3. Если нужно изменить - по ссылке

```
void modifyVector(std::vector<int>& vec) { /* ... */ }
```

// 4. Необязательные параметры - указатель с nullptr по умолчанию

```
void findValue(const std::vector<int>& data, int* foundIndex = nullptr) {
    if (foundIndex) *foundIndex = 42;
}
```

// 5. Выходные параметры - по ссылке

```
void getMinMax(const std::vector<int>& data, int& min, int& max) {
    min = data[0];
    max = data[0];
    // ...
}
```

// 6. Множество выходных параметров - через структуру

```
struct Result {
    int min, max, sum;
};
```

```
Result calculateStats(const std::vector<int>& data) {
    return {data[0], data[0], 0}; // C++17 structured binding
}
```

// НЕПРАВИЛЬНО:

// 1. Большой объект по значению (неэффективно)

```
// void badExample1(std::vector<int> data) {} // КОПИРОВАНИЕ!
```

// 2. Неинтуитивный интерфейс

```
// void badExample2(int& inputOutput) {} // Непонятно, входной или выходной
```

// 3. Указатель без проверки на nullptr

```
// void badExample3(int* ptr) { *ptr = 10; } // ОПАСНО!
```

```
int main() {
```

```
    // Пример использования
```

```
    std::vector<int> numbers = {1, 2, 3, 4, 5};
```

```
    // Простое чтение
```

```
    // printVector(numbers); // Должно принимать const std::vector<int>&
```

```
// Модификация
modifyVector(numbers);

// Получение нескольких результатов
int min, max;
getMinMax(numbers, min, max);

// Использование структуры для возврата
Result stats = calculateStats(numbers);

return 0;
}
```

Итоговые правила выбора:

1. Используйте передачу по значению для:

- Встроенных типов (int, double, char, bool)
- Маленьких структур (меньше 2-3 указателей)
- Перечислений (enum)

2. Используйте передачу по константной ссылке для:

- Больших объектов (строки, векторы, массивы)
- Классов и структур
- Когда нужно только читать данные

3. Используйте передачу по ссылке для:

- Выходных параметров
- Когда нужно изменить оригинальный объект
- Для экономии памяти при работе с большими объектами

4. Используйте указатели для:

- Необязательных параметров (с nullptr по умолчанию)
- Низкоуровневого кода
- Совместимости с C API

5. Используйте умные указатели для:

- Владения ресурсами
- Динамически выделяемой памяти
- Полиморфных объектов Подробное пошаговое объяснение работы программы

Давайте разберем **каждую строку** этой программы и поймем, **что происходит в памяти**.

Исходный код с нумерацией строк:

```
#include <iostream>

// 1. Простые типы по значению
void simpleByValue(int num, double price, char grade) {
    // Все параметры - локальные копии
    num = 100;           // Меняется только копия
    price = 99.99;       // Меняется только копия
    grade = 'A';         // Меняется только копия

    std::cout << "Внутри функции: " << num << ", " << price << ", " << gr
}

// 2. Указатель по значению
void pointerByValue(int* ptr) {
    // ptr - копия указателя, указывает на те же данные
    *ptr = 100;          // Меняем данные по адресу
    ptr = nullptr;       // Меняем только копию указателя
}

// 3. Массив по значению (на самом деле передается как указатель!)
void arrayByValue(int arr[5]) { // Размер игнорируется!
    // arr на самом деле int* - указатель на первый элемент
    arr[0] = 100;         // Меняет оригинальный массив!
}

// 4. Структура по значению (полное копирование)
struct Point {
    int x, y;
};

void structByValue(Point p) {
    p.x = 100;            // Меняется только копия
    p.y = 200;
}

int main() {
    int a = 5;
    double b = 10.5;
    char c = 'B';

    std::cout << "До вызова: " << a << ", " << b << ", " << c << std::enc
    simpleByValue(a, b, c);
    std::cout << "После вызова: " << a << ", " << b << ", " << c << std::
```



```

// Пример с указателем
int value = 50;
int* ptr = &value;
pointerByValue(ptr);
std::cout << "value после pointerByValue: " << value << std::endl;

// Пример с массивом
int arr[5] = {1, 2, 3, 4, 5};
arrayByValue(arr);
std::cout << "arr[0] после arrayByValue: " << arr[0] << std::endl;

// Пример со структурой
Point point = {10, 20};
structByValue(point);
std::cout << "point после structByValue: (" << point.x << ", " << poi

return 0;
}

```

Шаг 1: Выполнение main() - начало

Строки 31-33:

```

int main() {
    int a = 5;
    double b = 10.5;
    char c = 'B';
}

```

Что происходит в памяти:

СТЕК (stack) main():

a = 5	← int (4 байта)
b = 10.5	← double (8 байт)
c = 'B'	← char (1 байт)

Визуализация:

- a → ячейка памяти с адресом, например, 0x7ffd1234
- b → ячейка памяти с адресом, например, 0x7ffd1238
- c → ячейка памяти с адресом, например, 0x7ffd1240

Шаг 2: Вывод исходных значений

Строка 35:

```
std::cout << "До вызова: " << a << ", " << b << ", " << c << std::endl;
```

Результат:

До вызова: 5, 10.5, B

Шаг 3: Вызов simpleByValue(a, b, c)

Строка 36:

```
simpleByValue(a, b, c);
```

Что происходит ПРИ ВЫЗОВЕ функции:

1. Создается новый фрейм стека для функции `simpleByValue`
2. Значения `a`, `b`, `c` КОПИРУЮТСЯ в новые переменные внутри функции

ДО ВЫЗОВА:

main() стек	
a = 5	
b = 10.5	
c = 'B'	

ВО ВРЕМЯ ВЫЗОВА:

simpleByValue() стек	
num = 5	← КОПИЯ значения a
price = 10.5	← КОПИЯ значения b
grade = 'B'	← КОПИЯ значения c
main() стек (остается!)	
a = 5	← ОРИГИНАЛ не меняется!
b = 10.5	← ОРИГИНАЛ не меняется!

c = 'B'	← ОРИГИНАЛ не меняется!
---------	-------------------------

Шаг 4: Выполнение simpleByValue()

Строки 4-7:

```
void simpleByValue(int num, double price, char grade) {
    // Все параметры - локальные копии
    num = 100;           // Меняется только копия
    price = 99.99;       // Меняется только копия
    grade = 'A';         // Меняется только копия
}
```

Что происходит в памяти функции:

ПЕРЕД изменением:

num = 5	simpleByValue() стек
price = 10.5	
grade = 'B'	

ПОСЛЕ изменения:

num = 100	simpleByValue() стек
price = 99.99	
grade = 'A'	

← ИЗМЕНИЛАСЬ КОПИЯ!

← ИЗМЕНИЛАСЬ КОПИЯ!

← ИЗМЕНИЛАСЬ КОПИЯ!

НО в main() стеке:

a = 5	main() стек
b = 10.5	
c = 'B'	

← ОРИГИНАЛ БЕЗ ИЗМЕНЕНИЙ!

← ОРИГИНАЛ БЕЗ ИЗМЕНЕНИЙ!

← ОРИГИНАЛ БЕЗ ИЗМЕНЕНИЙ!

Строка 8:

```
std::cout << "Внутри функции: " << num << ", " << price << ", " << grade
```

Результат:

Внутри функции: 100, 99.99, A

Шаг 5: Возврат из simpleByValue()**Что происходит при возврате:**

1. Уничтожается стековый фрейм функции `simpleByValue`
2. Все локальные переменные (`num`, `price`, `grade`) уничтожаются
3. Управление возвращается в `main()`

ПОСЛЕ возврата:

a = 5	main() стек (как и было!)
b = 10.5	
c = 'B'	

Шаг 6: Вывод после возврата**Строка 37:**

```
std::cout << "После вызова: " << a << ", " << b << ", " << c << std::end
```

Результат:

После вызова: 5, 10.5, B

Вывод: Оригинальные значения НЕ изменились!

Шаг 7: Работа с указателями**Строки 40-41:**

```
int value = 50;
int* ptr = &value;
```

Что происходит в памяти:

main() стек:

value = 50	← переменная типа int
ptr = 0x...	← указатель (хранит АДРЕС value)

Адресная арифметика:

- `&value` → возвращает адрес переменной value
- `ptr` хранит этот адрес
- `*ptr` дает доступ к значению по этому адресу

Визуализация:

value: [50] ← значение

↑

ptr: [адрес value] ← хранит "ссылку" на value

Шаг 8: Вызов `pointerByValue(ptr)`

Строка 42:

```
pointerByValue(ptr);
```

Что происходит при вызове:

1. Создается копия указателя ptr
2. Обе копии указывают на ОДНУ И ТУ ЖЕ память

ДО вызова в main():

value: [50]

↑

ptr: [адрес value]

ВО ВРЕМЯ вызова:

main() стек:

value: [50]

pointerByValue() стек:

ptr_copy: [адрес value]

↑
↑
 ptr: [адрес value] (такой же адрес!)

Шаг 9: Выполнение pointerByValue()

Строки 11-14:

```
void pointerByValue(int* ptr) {
    // ptr - копия указателя, указывает на те же данные
    *ptr = 100;           // Меняем данные по адресу
    ptr = nullptr;       // Меняем только копию указателя
}
```

Пошагово:

1. ***ptr = 100;** - разыменовываем указатель

До: value: [50]

После: value: [100] ← ИЗМЕНИЛОСЬ оригинальное значение!

Почему изменилось? Потому что обе копии указателя (ptr в main и ptr в функции) содержат ОДИН И ТОТ ЖЕ АДРЕС

2. **ptr = nullptr;** - меняем саму копию указателя

В функции: ptr_copy: [nullptr] ← изменилась копия

В main(): ptr: [адрес value] ← оригинальный указатель не изменился



Шаг 10: Возврат и вывод

Строка 43:

```
std::cout << "value после pointerByValue: " << value << std::endl;
```

Результат:

```
value после pointerByValue: 100
```

Вывод: Значение изменилось через указатель!

Шаг 11: Работа с массивом

Строка 46:

```
int arr[5] = {1, 2, 3, 4, 5};
```

Что происходит в памяти:

main() стек:

arr[0] = 1	
arr[1] = 2	
arr[2] = 3	
arr[3] = 4	
arr[4] = 5	

Важно: имя массива `arr` - это указатель на первый элемент!
`arr == &arr[0]`

Шаг 12: Вызов arrayByValue(arr)

Строка 47:

```
arrayByValue(arr);
```

Ключевой момент:

- В C/C++ массивы НЕ передаются по значению (не копируются)
- Передается указатель на первый элемент
- `arrayByValue(int arr[5])` компилятор преобразует в `arrayByValue(int* arr)`

Передача массива:

main(): `arr` → [1][2][3][4][5]

↑

функция получает: `int* arr = адрес arr[0]`

Шаг 13: Выполнение `arrayByValue()`

Строки 17-20:

```
void arrayByValue(int arr[5]) { // Размер игнорируется!
    // arr на самом деле int* - указатель на первый элемент
    arr[0] = 100;           // Меняет оригинальный массив!
}
```

Что происходит:

```
arr[0] = 100;
// Эквивалентно:
*(arr + 0) = 100;
// И так как arr содержит адрес arr[0] из main():
*(адрес_первого_элемента) = 100;
```

До: `main(): [1][2][3][4][5]`
 После: `main(): [100][2][3][4][5]` ← ИЗМЕНИЛСЯ оригинал!

Шаг 14: Вывод после `arrayByValue`

Строка 48:

```
std::cout << "arr[0] после arrayByValue: " << arr[0] << std::endl;
```

Результат:

```
arr[0] после arrayByValue: 100
```

Шаг 15: Работа со структурой

Строки 23-27 и 51:


```

struct Point {
    int x, y;
};

Point point = {10, 20};

```

Что происходит:

main() стек:

point.x = 10	
point.y = 20	

Шаг 16: Вызов structByValue(point)

Строка 52:

```
structByValue(point);
```

Что происходит:

1. Создается ПОЛНАЯ КОПИЯ структуры Point
2. Копируются ВСЕ поля структуры

ДО вызова:

main(): point {x=10, y=20}

ВО ВРЕМЯ вызова:

main(): point {x=10, y=20} ← оригинал

функция: p {x=10, y=20} ← ПОЛНАЯ КОПИЯ (8 байт)

Шаг 17: Выполнение structByValue()

Строки 29-32:

```

void structByValue(Point p) {
    p.x = 100;           // Меняется только копия
    p.y = 200;
}

```

В функции: `p {x=100, y=200}` ← изменилась КОПИЯ
 В `main()`: `point {x=10, y=20}` ← оригинал НЕ изменился

Шаг 18: Вывод после `structByValue`

Строка 53:

```
std::cout << "point после structByValue: (" << point.x << ", " << point.y << endl;
```

Результат:

```
point после structByValue: (10, 20)
```

Полный вывод программы:

```
До вызова: 5, 10.5, B
Внутри функции: 100, 99.99, A
После вызова: 5, 10.5, B
value после pointerByValue: 100
arr[0] после arrayByValue: 100
point после structByValue: (10, 20)
```

Ключевые выводы:

1. **Простые типы по значению** → создается копия, оригинал не меняется
2. **Указатель по значению** → копируется сам указатель, но обе копии указывают на одни данные
3. **Массив как параметр** → фактически передается указатель, меняется оригинал
4. **Структура по значению** → полное копирование, оригинал не меняется

Визуальная схема памяти:

`main()` стек до всех вызовов:

a = 5	← int
b = 10.5	← double

c = 'B'	← char
value = 50	← int
ptr → value	← указатель
arr[0..4] = 1,2,3,4,5	← массив
point.x = 10, y = 20	← структура

ПОСЛЕ всех вызовов:

a = 5	← НЕ изменилось
b = 10.5	← НЕ изменилось
c = 'B'	← НЕ изменилось
value = 100	← ИЗМЕНИЛОСЬ через указатель
ptr → value	← указатель прежний
arr[0] = 100 arr[1..4] = 2,3,4,5	← ИЗМЕНИЛОСЬ через массив-указатель
point.x = 10, y = 20	← НЕ изменилось

Полный код программы с комментариями и выводом

```
#include <iostream>

// 1. Функция с параметрами по значению (простые типы)
// Параметры num, price, grade - это КОПИИ переданных значений
void simpleByValue(int num, double price, char grade) {
    // Все изменения происходят только с локальными копиями
    num = 100;           // Изменяем копию переменной a из main()
    price = 99.99;       // Изменяем копию переменной b из main()
    grade = 'A';         // Изменяем копию переменной c из main()

    // Выводим измененные значения (только копий)
    std::cout << "Внутри функции simpleByValue: "
```

```

        << num << ", " << price << ", " << grade << std::endl;
    }

// 2. Функция с указателем как параметром по значению
// Указатель ptr - это КОПИЯ указателя из main()
void pointerByValue(int* ptr) {
    // ptr - копия указателя, но обе копии указывают на одни данные

    // Меняем данные по адресу, на который указывает ptr
    // Так как ptr и оригинальный указатель из main() указывают
    // на одну и ту же память, меняется оригинальное значение
    *ptr = 100;

    // Меняем саму копию указателя (делаем её nullptr)
    // Это НЕ влияет на оригинальный указатель в main()
    ptr = nullptr;
}

// 3. Функция с массивом как параметром
// Важно: в C/C++ массивы передаются как указатели!
// Синтаксис int arr[5] компилятор преобразует в int* arr
void arrayByValue(int arr[5]) {
    // Размер [5] игнорируется компилятором!
    // arr - это указатель на первый элемент массива

    // Меняем первый элемент массива через указатель
    // Так как arr указывает на оригинальный массив из main(),
    // изменения будут видны в main()
    arr[0] = 100;
}

// 4. Объявление структуры Point
struct Point {
    int x; // Координата X
    int y; // Координата Y
};

// 5. Функция со структурой как параметром по значению
// Структура p - это ПОЛНАЯ КОПИЯ структуры из main()
void structByValue(Point p) {
    // Изменяем поля копии структуры
    p.x = 100; // Изменяем копию поля x
    p.y = 200; // Изменяем копию поля y

    // Эти изменения НЕ влияют на оригинальную структуру в main()
}

// Главная функция программы
int main() {
    // ===== ЧАСТЬ 1: Простые типы по значению =====

```

```

std::cout << "=== Часть 1: Простые типы по значению ===" << std::endl

int a = 5;           // Объявление и инициализация целочисленной пере
double b = 10.5;     // Объявление и инициализация переменной с плава
char c = 'B';        // Объявление и инициализация символьной перемен

// Выводим значения ДО вызова функции
std::cout << "До вызова simpleByValue:" << std::endl;
std::cout << "a = " << a << ", b = " << b << ", c = " << c << std::er

// Вызываем функцию, передавая параметры ПО ЗНАЧЕНИЮ
// Значения a, b, c копируются в параметры функции
simpleByValue(a, b, c);

// Выводим значения ПОСЛЕ вызова функции
// Оригинальные значения a, b, c НЕ изменились!
std::cout << "После вызова simpleByValue:" << std::endl;
std::cout << "a = " << a << ", b = " << b << ", c = " << c << std::er

// ===== ЧАСТЬ 2: Указатели по значению =====
std::cout << "\n=== Часть 2: Указатели по значению ===" << std::endl;

int value = 50;      // Создаем переменную value
int* ptr = &value;   // Создаем указатель ptr и присваиваем ему адрес

std::cout << "До вызова pointerByValue:" << std::endl;
std::cout << "value = " << value << std::endl;
std::cout << "*ptr = " << *ptr << std::endl;

// Вызываем функцию, передавая указатель ПО ЗНАЧЕНИЮ
// Указатель ptr копируется, но обе копии указывают на value
pointerByValue(ptr);

std::cout << "После вызова pointerByValue:" << std::endl;
std::cout << "value = " << value << " (изменилось через указатель!)"

// Проверяем, что указатель ptr НЕ стал nullptr
// (в функции менялась только копия указателя)
if (ptr != nullptr) {
    std::cout << "*ptr = " << *ptr << " (указатель не изменился)" <<
} else {
    std::cout << "ptr = nullptr (указатель изменился)" << std::endl;
}

// ===== ЧАСТЬ 3: Массивы как параметры =====
std::cout << "\n=== Часть 3: Массивы как параметры ===" << std::endl;

// Создаем и инициализируем массив из 5 элементов
int arr[5] = {1, 2, 3, 4, 5};

```

```

std::cout << "До вызова arrayByValue:" << std::endl;
std::cout << "arr[0] = " << arr[0] << std::endl;
std::cout << "Весь массив: ";
for (int i = 0; i < 5; i++) {
    std::cout << arr[i] << " ";
}
std::cout << std::endl;

// Вызываем функцию, передавая массив
// Массив передается как указатель на первый элемент
arrayByValue(arr);

std::cout << "После вызова arrayByValue:" << std::endl;
std::cout << "arr[0] = " << arr[0] << " (изменился!)" << std::endl;
std::cout << "Весь массив: ";
for (int i = 0; i < 5; i++) {
    std::cout << arr[i] << " ";
}
std::cout << std::endl;

// ===== ЧАСТЬ 4: Структуры по значению =====
std::cout << "\n=== Часть 4: Структуры по значению ===" << std::endl;

// Создаем и инициализируем структуру Point
Point point = {10, 20}; // point.x = 10, point.y = 20

std::cout << "До вызова structByValue:" << std::endl;
std::cout << "point.x = " << point.x << ", point.y = " << point.y <<

// Вызываем функцию, передавая структуру ПО ЗНАЧЕНИЮ
// Создается полная копия структуры
structByValue(point);

std::cout << "После вызова structByValue:" << std::endl;
std::cout << "point.x = " << point.x << ", point.y = " << point.y;
std::cout << " (не изменились!)" << std::endl;

// ===== ВЫВОДЫ И СРАВНЕНИЕ =====
std::cout << "\n=== Итоговые выводы ===" << std::endl;
std::cout << "1. Простые типы по значению: ОРИГИНАЛЫ НЕ МЕНЯЮТСЯ" <<
std::cout << "2. Указатели по значению: ДАННЫЕ ПО УКАЗАТЕЛЮ МЕНЯЮТСЯ,
    << "сам указатель - НЕТ" << std::endl;
std::cout << "3. Массивы как параметры: ОРИГИНАЛЬНЫЙ МАССИВ МЕНЯЕТСЯ
    << "(передается как указатель)" << std::endl;
std::cout << "4. Структуры по значению: ОРИГИНАЛ НЕ МЕНЯЕТСЯ "
    << "(создается полная копия)" << std::endl;

return 0; // Завершение программы с кодом 0 (успех)
}

```

Ожидаемый вывод программы:

=== Часть 1: Простые типы по значению ===

До вызова simpleByValue:

a = 5, b = 10.5, c = B

Внутри функции simpleByValue: 100, 99.99, A

После вызова simpleByValue:

a = 5, b = 10.5, c = B

=== Часть 2: Указатели по значению ===

До вызова pointerByValue:

value = 50

*ptr = 50

После вызова pointerByValue:

value = 100 (изменилось через указатель!)

*ptr = 100 (указатель не изменился)

=== Часть 3: Массивы как параметры ===

До вызова arrayByValue:

arr[0] = 1

Весь массив: 1 2 3 4 5

После вызова arrayByValue:

arr[0] = 100 (изменился!)

Весь массив: 100 2 3 4 5

=== Часть 4: Структуры по значению ===

До вызова structByValue:

point.x = 10, point.y = 20

После вызова structByValue:

point.x = 10, point.y = 20 (не изменились!)

=== Итоговые выводы ===

1. Простые типы по значению: ОРИГИНАЛЫ НЕ МЕНЯЮТСЯ

2. Указатели по значению: ДАННЫЕ ПО УКАЗАТЕЛЮ МЕНЯЮТСЯ, сам указатель - НЕТ

3. Массивы как параметры: ОРИГИНАЛЬНЫЙ МАССИВ МЕНЯЕТСЯ (передается как указатель)

4. Структуры по значению: ОРИГИНАЛ НЕ МЕНЯЕТСЯ (создается полная копия)



Ключевые моменты для понимания:

1. Простые типы по значению

```
void func(int a) { a = 100; } // Меняется только копия
int main() {
    int x = 5;
```

```
    func(x);  // x останется 5
}
```

2. Указатели по значению

```
void func(int* p) {
    *p = 100;    // Меняем данные по адресу
    p = nullptr; // Меняем только копию указателя
}
int main() {
    int x = 5;
    int* ptr = &x;
    func(ptr);  // x станет 100, но ptr не станет nullptr
}
```

3. Массивы как параметры

```
void func(int arr[]) { // То же что int* arr
    arr[0] = 100;  // Меняет оригинальный массив!
}
int main() {
    int arr[3] = {1, 2, 3};
    func(arr);  // arr[0] станет 100
}
```

4. Структуры по значению

```
struct Point { int x, y; };
void func(Point p) { p.x = 100; } // Меняет только копию
int main() {
    Point p = {10, 20};
    func(p);  // p.x останется 10
}
```

Как скомпилировать и запустить:

1. **Сохраните код** в файл `memory_demo.cpp`
2. **Скомпилируйте** командой:

```
g++ -o memory_demo memory_demo.cpp
```


или

```
g++ -std=c++11 -o memory_demo memory_demo.cpp
```

3. **Запустите** программу:

```
./memory_demo
```

Эта программа наглядно демонстрирует все основные способы передачи параметров в C++ и их влияние на оригинальные данные.

Все виды синтаксиса параметров функций в C++

1. Параметры по значению (самые простые)

Базовый синтаксис

```
тип имя_функции(тип параметр1, тип параметр2, ...)
```

Примеры и задачи

Задача 1.1: Функция для вычисления площади прямоугольника

```
#include <iostream>
using namespace std;

// Параметры по значению - создаются копии
double rectangleArea(double width, double height) {
    return width * height;
}

int main() {
    double w = 5.5, h = 3.2;

    // Передаются КОПИИ w и h
    double area = rectangleArea(w, h);

    cout << "Ширина: " << w << endl;
    cout << "Высота: " << h << endl;
    cout << "Площадь: " << area << endl;
```

```

    // w и h не изменяются функцией
    cout << "После вызова функции w = " << w << ", h = " << h << endl;

    return 0;
}

// Плюсы параметров по значению:
// 1. Простота понимания
// 2. Безопасность (оригинальные данные защищены)
// Минусы:
// 1. Накладные расходы на копирование больших объектов

```

Задача 1.2: Функция swap, которая НЕ работает (демонстрация ограничения)

```

#include <iostream>
using namespace std;

// НЕПРАВИЛЬНАЯ функция swap - параметры по значению
void wrongSwap(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
    cout << "Внутри wrongSwap: a = " << a << ", b = " << b << endl;
}

int main() {
    int x = 10, y = 20;

    cout << "До wrongSwap: x = " << x << ", y = " << y << endl;

    wrongSwap(x, y); // Передаются копии!

    cout << "После wrongSwap: x = " << x << ", y = " << y << endl;
    // x и y НЕ поменялись местами!

    return 0;
}

/*
Вывод:
До wrongSwap: x = 10, y = 20
Внутри wrongSwap: a = 20, b = 10
После wrongSwap: x = 10, y = 20
*/

```

2. Параметры по ссылке (Reference Parameters)

Синтаксис

тип имя_функции(тип& параметр1, тип& параметр2, ...)

Примеры и задачи

Задача 2.1: Правильная функция swap

```
#include <iostream>
using namespace std;

// Правильная функция swap - параметры по ссылке
void swap(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}

// Функция для увеличения значения
void increment(int& x, int step = 1) {
    x += step; // Изменяет оригинальную переменную
}

// Функция, возвращающая несколько значений через ссылки
void getMinMax(const int arr[], int size, int& min, int& max) {
    if(size == 0) return;

    min = arr[0];
    max = arr[0];

    for(int i = 1; i < size; i++) {
        if(arr[i] < min) min = arr[i];
        if(arr[i] > max) max = arr[i];
    }
}

int main() {
    // Пример 1: swap
    int x = 10, y = 20;
    cout << "До swap: x = " << x << ", y = " << y << endl;
    swap(x, y);
    cout << "После swap: x = " << x << ", y = " << y << endl;

    // Пример 2: increment
    int counter = 5;
    cout << "\nДо increment: counter = " << counter << endl;
    increment(counter, 3);
    cout << "После increment(counter, 3): counter = " << counter << endl;
```

```

increment(counter); // используем значение по умолчанию
cout << "После increment(counter): counter = " << counter << endl;

// Пример 3: возврат нескольких значений
int numbers[] = {3, 7, 2, 9, 1, 4, 6, 8, 5};
int minValue, maxValue;

getMinMax(numbers, 9, minValue, maxValue);

cout << "\nМинимальное значение: " << minValue << endl;
cout << "Максимальное значение: " << maxValue << endl;

return 0;
}

```

Задача 2.2: Модификация элементов массива

```

#include <iostream>
using namespace std;

// Умножение всех элементов массива на множитель
void multiplyArray(int arr[], int size, int multiplier) {
    // arr[] фактически передается как указатель (ссылка на первый элемент)
    for(int i = 0; i < size; i++) {
        arr[i] *= multiplier; // Изменяет оригинальный массив
    }
}

// Заполнение массива последовательными числами
void fillArray(int arr[], int size, int start = 0) {
    for(int i = 0; i < size; i++) {
        arr[i] = start + i;
    }
}

void printArray(const int arr[], int size) {
    for(int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int main() {
    const int SIZE = 5;
    int myArray[SIZE];

    // Заполняем массив
    fillArray(myArray, SIZE, 10);
    cout << "Исходный массив: ";
}

```

```
printArray(myArray, SIZE);

// Умножаем все элементы на 2
multiplyArray(myArray, SIZE, 2);
cout << "После умножения на 2: ";
printArray(myArray, SIZE);

// Меняем только часть массива
multiplyArray(myArray + 2, 2, 10); // Умножаем элементы с индексами
cout << "После умножения элементов 2-3 на 10: ";
printArray(myArray, SIZE);

return 0;
}
```

3. Константные параметры (Const Parameters)

Синтаксис

```
// Константный параметр по значению (редко используется)
тип имя_функции(const тип параметр)

// Константный параметр по ссылке (рекомендуется для больших объектов)
тип имя_функции(const тип& параметр)
```

Примеры и задачи

Задача 3.1: Безопасная работа с большими объектами

```
#include <iostream>
#include <string>
using namespace std;

struct Student {
    string name;
    int age;
    double gpa;
    string courses[10];
    int courseCount;
};

// БЕЗОПАСНО: константная ссылка
// Не создает копию и гарантирует неизменность
void printStudent(const Student& student) {
    cout << "Студент: " << student.name << endl;
```

```
cout << "Возраст: " << student.age << endl;
cout << "Средний балл: " << student.gpa << endl;

// Нельзя изменить student!
// student.age = 20; // ОШИБКА компиляции
}

// НЕЭФФЕКТИВНО: параметр по значению
// Создает полную копию структуры
void printStudentInefficient(Student student) {
    cout << "Имя: " << student.name << endl;
    // Здесь тоже нельзя изменить оригинал,
    // но создается ненужная копия
}

// ОПАСНО: обычная ссылка
// Может случайно изменить данные
void printStudentUnsafe(Student& student) {
    cout << "Студент: " << student.name << endl;
    student.age++; // Неожиданный побочный эффект!
    cout << "Теперь возраст: " << student.age << endl;
}

// Сравнение двух студентов (обе ссылки константные)
bool compareStudents(const Student& s1, const Student& s2) {
    return s1.gpa > s2.gpa; // Сравниваем по GPA
}

int main() {
    Student alice = {"Алиса", 20, 4.5, {"Математика", "Физика"}, 2};
    Student bob = {"Боб", 21, 4.2, {"Программирование", "История"}, 2};

    cout << "=== Безопасная версия ===" << endl;
    printStudent(alice);

    cout << "\n=== Небезопасная версия ===" << endl;
    cout << "Возраст Алисы до вызова: " << alice.age << endl;
    printStudentUnsafe(alice);
    cout << "Возраст Алисы после вызова: " << alice.age << endl;

    cout << "\n=== Сравнение студентов ===" << endl;
    if(compareStudents(alice, bob)) {
        cout << alice.name << " имеет лучший средний балл" << endl;
    } else {
        cout << bob.name << " имеет лучший средний балл" << endl;
    }

    return 0;
}
```

Задача 3.2: Константные параметры с массивами

```
#include <iostream>
using namespace std;

// Константный массив - гарантия, что функция не изменит данные
double calculateAverage(const int grades[], int count) {
    if(count == 0) return 0.0;

    int sum = 0;
    for(int i = 0; i < count; i++) {
        sum += grades[i];
        // grades[i] = 0; // ОШИБКА! Массив константный
    }

    return static_cast<double>(sum) / count;
}

// Поиск элемента в константном массиве
int findIndex(const int arr[], int size, int value) {
    for(int i = 0; i < size; i++) {
        if(arr[i] == value) {
            return i;
        }
    }
    return -1; // не найден
}

// Константная ссылка на массив фиксированного размера
void printFixedArray(const int (&arr)[5]) {
    // Этот синтаксис гарантирует, что массив имеет именно 5 элементов
    for(int i = 0; i < 5; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int main() {
    int grades[] = {5, 4, 3, 5, 4, 5};
    int size = 6;

    cout << "Оценки: ";
    for(int i = 0; i < size; i++) cout << grades[i] << " ";
    cout << endl;

    double avg = calculateAverage(grades, size);
    cout << "Средний балл: " << avg << endl;

    // Поиск оценки
    int searchGrade = 3;
```

```

int index = findIndex(grades, size, searchGrade);
if(index != -1) {
    cout << "Оценка " << searchGrade << " найдена на позиции " << inc
}

// Массив фиксированного размера
int fixedArray[5] = {1, 2, 3, 4, 5};
cout << "Массив фиксированного размера: ";
printFixedArray(fixedArray);

return 0;
}

```

4. Параметры-указатели (Pointer Parameters)

Синтаксис

тип имя_функции(тип* параметр)

Примеры и задачи

Задача 4.1: Работа с указателями как параметрами

```

#include <iostream>
using namespace std;

// Принимает указатель на int
void incrementByPointer(int* ptr) {
    if(ptr != nullptr) { // Всегда проверяем указатель!
        (*ptr)++; // Декрементируем значение по указателю
    }
}

// Функция для обмена значений через указатели
void swapPointers(int* a, int* b) {
    if(a != nullptr && b != nullptr) {
        int temp = *a;
        *a = *b;
        *b = temp;
    }
}

// Возврат результата через указатель
void getStats(const int arr[], int size, int* min, int* max, double* avg)
    if(size == 0 || min == nullptr || max == nullptr || avg == nullptr) {

```



```

        return;
    }

    *min = arr[0];
    *max = arr[0];
    int sum = 0;

    for(int i = 0; i < size; i++) {
        if(arr[i] < *min) *min = arr[i];
        if(arr[i] > *max) *max = arr[i];
        sum += arr[i];
    }

    *avg = static_cast<double>(sum) / size;
}

// Сравнение указателей и ссылок
void demoPointersVsReferences() {
    int x = 10;
    int y = 20;

    int* ptrX = &x; // указатель
    int& refY = y;   // ссылка

    cout << "x = " << x << ", y = " << y << endl;
    cout << "*ptrX = " << *ptrX << ", refY = " << refY << endl;

    // Изменение через указатель
    *ptrX = 100;
    cout << "После *ptrX = 100: x = " << x << endl;

    // Изменение через ссылку
    refY = 200;
    cout << "После refY = 200: y = " << y << endl;

    // Указатель можно перенаправить
    ptrX = &y;
    cout << "После ptrX = &y: *ptrX = " << *ptrX << endl;

    // Ссылку перенаправить НЕЛЬЗЯ
    // refY = x; // Это присваивание значения, не перенаправление ссылки
}

int main() {
    int a = 5, b = 10;

    // Работа с incrementByPointer
    cout << "До incrementByPointer: a = " << a << endl;
    incrementByPointer(&a); // Передаем адрес переменной
    cout << "После incrementByPointer: a = " << a << endl;
}

```

```

// Работа с swapPointers
cout << "\nДо swapPointers: a = " << a << ", b = " << b << endl;
swapPointers(&a, &b);
cout << "После swapPointers: a = " << a << ", b = " << b << endl;

// Получение статистики через указатели
int numbers[] = {3, 7, 2, 9, 1, 4, 6};
int minVal, maxVal;
double average;

getStats(numbers, 7, &minVal, &maxVal, &average);

cout << "\nСтатистика массива:" << endl;
cout << "Минимум: " << minVal << endl;
cout << "Максимум: " << maxVal << endl;
cout << "Среднее: " << average << endl;

cout << "\n=== Демонстрация указателей vs ссылок ===" << endl;
demoPointersVsReferences();

return 0;
}

```

Задача 4.2: Динамические массивы и указатели

```

#include <iostream>
#include <cstdlib> // для rand()
using namespace std;

// Создание и заполнение динамического массива
int* createRandomArray(int size, int minVal, int maxVal) {
    if(size <= 0) return nullptr;

    int* arr = new int[size];

    for(int i = 0; i < size; i++) {
        arr[i] = minVal + rand() % (maxVal - minVal + 1);
    }

    return arr; // Возвращаем указатель
}

// Изменение размера массива (realloc аналог)
int* resizeArray(int* arr, int oldSize, int newSize) {
    if(newSize <= 0) {
        delete[] arr;
        return nullptr;
    }
}

```

```
int* newArr = new int[newSize];

// Копируем старые данные
int copySize = (oldSize < newSize) ? oldSize : newSize;
for(int i = 0; i < copySize; i++) {
    newArr[i] = arr[i];
}

// Освобождаем старую память
delete[] arr;

return newArr;
}

// Функция принимает указатель на указатель
void allocateMatrix(int*** matrix, int rows, int cols) {
    *matrix = new int*[rows];
    for(int i = 0; i < rows; i++) {
        (*matrix)[i] = new int[cols];
    }
}

void freeMatrix(int** matrix, int rows) {
    for(int i = 0; i < rows; i++) {
        delete[] matrix[i];
    }
    delete[] matrix;
}

int main() {
    srand(time(0)); // Инициализация генератора случайных чисел

    // Пример 1: Динамический массив
    int size = 5;
    int* dynamicArray = createRandomArray(size, 1, 100);

    cout << "Исходный массив: ";
    for(int i = 0; i < size; i++) {
        cout << dynamicArray[i] << " ";
    }
    cout << endl;

    // Изменяем размер массива
    int newSize = 8;
    dynamicArray = resizeArray(dynamicArray, size, newSize);

    cout << "После увеличения до " << newSize << " элементов: ";
    for(int i = 0; i < newSize; i++) {
        cout << dynamicArray[i] << " ";
    }
}
```

```

    }
    cout << endl;

    delete[] dynamicArray; // Не забываем освободить память!

    // Пример 2: Двумерный массив (матрица)
    int** matrix;
    int rows = 3, cols = 4;

    allocateMatrix(&matrix, rows, cols);

    // Заполняем матрицу
    for(int i = 0; i < rows; i++) {
        for(int j = 0; j < cols; j++) {
            matrix[i][j] = i * 10 + j;
        }
    }

    cout << "\nМатрица " << rows << "x" << cols << ":" << endl;
    for(int i = 0; i < rows; i++) {
        for(int j = 0; j < cols; j++) {
            cout << matrix[i][j] << "\t";
        }
        cout << endl;
    }

    freeMatrix(matrix, rows);

    return 0;
}

```

5. Смешанные параметры и продвинутые случаи

Задача 5.1: Все виды параметров в одной функции

```

#include <iostream>
using namespace std;

// Функция с разными типами параметров:
// 1. По значению (value)
// 2. По ссылке (reference)
// 3. Константная ссылка (const reference)
// 4. Указатель (pointer)
// 5. Константный указатель (const pointer)
// 6. Указатель на константу (pointer to const)
void complexFunction(
    int byValue,           // 1. Простая копия

```

```

int& byRef,           // 2. Ссылка (можно изменить)
const int& byConstRef, // 3. Константная ссылка (только чтение)
int* byPointer,       // 4. Указатель (можно изменить значение)
int* const constPtr,  // 5. Константный указатель (адрес не меняется)
const int* ptrToConst // 6. Указатель на константу
) {
    cout << "\n=== Внутри complexFunction ===" << endl;

    // 1. Параметр по значению
    byValue = 999; // Меняем копию, оригинал не меняется
    cout << "byValue (копия): " << byValue << endl;

    // 2. Ссылка
    byRef = 888; // Меняем оригинал
    cout << "byRef (изменили оригинал): " << byRef << endl;

    // 3. Константная ссылка
    // byConstRef = 777; // ОШИБКА! Нельзя изменить
    cout << "byConstRef (только чтение): " << byConstRef << endl;

    // 4. Указатель
    if(byPointer != nullptr) {
        *byPointer = 666; // Меняем значение по указателю
        cout << "*byPointer (изменили через указатель): " << *byPointer <

        // Можно изменить сам указатель
        int temp = 100;
        byPointer = &temp;
        cout << "Перенаправили byPointer на temp: " << *byPointer << endl
    }

    // 5. Константный указатель
    if(constPtr != nullptr) {
        *constPtr = 555; // Можем изменить значение
        cout << "*constPtr (значение изменено): " << *constPtr << endl;

        // constPtr = &temp; // ОШИБКА! Нельзя изменить адрес
    }

    // 6. Указатель на константу
    // *ptrToConst = 444; // ОШИБКА! Нельзя изменить значение

    if(ptrToConst != nullptr) {
        cout << "*ptrToConst (только чтение): " << *ptrToConst << endl;

        // Можно изменить сам указатель
        int temp = 200;
        ptrToConst = &temp;
        cout << "Перенаправили ptrToConst на temp: " << *ptrToConst << er
    }
}

```

```

}

int main() {
    int a = 1, b = 2, c = 3, d = 4, e = 5, f = 6;

    cout << "До вызова функции:" << endl;
    cout << "a = " << a << ", b = " << b << ", c = " << c << endl;
    cout << "d = " << d << ", e = " << e << ", f = " << f << endl;

    complexFunction(a, b, c, &d, &e, &f);

    cout << "\nПосле вызова функции:" << endl;
    cout << "a (по значению) = " << a << " - не изменился" << endl;
    cout << "b (по ссылке) = " << b << " - изменился" << endl;
    cout << "c (конст. ссылка) = " << c << " - не изменился" << endl;
    cout << "d (указатель) = " << d << " - изменился" << endl;
    cout << "e (конст. указатель) = " << e << " - изменился" << endl;
    cout << "f (указ. на константу) = " << f << " - не изменился" << endl;

    return 0;
}

```

Задача 5.2: Функции с параметрами по умолчанию

```

#include <iostream>
#include <string>
using namespace std;

// Параметры по умолчанию ДОЛЖНЫ идти в конце
void printUserInfo(
    string name,
    int age = 0,                // параметр по умолчанию
    string city = "Не указан", // еще один
    bool showDetails = true    // и еще один
) {
    cout << "Имя: " << name << endl;

    if(showDetails) {
        cout << "Возраст: " << age << endl;
        cout << "Город: " << city << endl;
    }

    cout << "---" << endl;
}

// НЕВЕРНО! Параметры по умолчанию не могут быть в начале
// void badFunction(int a = 0, int b, int c) { // ОШИБКА!
// }

```

```
// Перегрузка с параметрами по умолчанию
double calculateArea(double radius) {
    return 3.14159 * radius * radius;
}

double calculateArea(double length, double width, bool isRectangle = true) {
    if(isRectangle) {
        return length * width;
    } else {
        // Для треугольника
        return 0.5 * length * width;
    }
}

int main() {
    // Разные способы вызова функции с параметрами по умолчанию
    printUserInfo("Анна", 25, "Москва", true);
    printUserInfo("Борис", 30, "Санкт-Петербург"); // showDetails = true
    printUserInfo("Виктор", 35); // city и showDetails по умолчанию
    printUserInfo("Галина"); // все параметры по умолчанию
    printUserInfo("Дмитрий", 40, "Казань", false); // не показывать детали

    cout << "\n=== Расчет площадей ===" << endl;
    cout << "Площадь круга радиусом 5: " << calculateArea(5) << endl;
    cout << "Площадь прямоугольника 4x6: " << calculateArea(4, 6) << endl;
    cout << "Площадь треугольника (основание 4, высота 6): "
        << calculateArea(4, 6, false) << endl;

    return 0;
}
```

6. Шаблоновые функции (Template Functions)

Задача 6.1: Обобщенные функции

```
#include <iostream>
using namespace std;

// Шаблоновая функция для обмена значений
template<typename T>
void templateSwap(T& a, T& b) {
    T temp = a;
    a = b;
    b = temp;
}

// Шаблоновая функция для поиска максимума
```

```

template<typename T>
T getMax(T a, T b) {
    return (a > b) ? a : b;
}

// Шаблонная функция для массива
template<typename T, int N>
void printArray(T (&arr)[N]) {
    cout << "Массив из " << N << " элементов: ";
    for(int i = 0; i < N; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

// Специализация шаблона для строк
template<>
void printArray<char>(char (&arr)[10]) {
    cout << "Строка: ";
    for(int i = 0; i < 10 && arr[i] != '\0'; i++) {
        cout << arr[i];
    }
    cout << endl;
}

int main() {
    // Работа с разными типами данных
    int x = 5, y = 10;
    cout << "До swap: x = " << x << ", y = " << y << endl;
    templateSwap(x, y);
    cout << "После swap: x = " << x << ", y = " << y << endl;

    double d1 = 3.14, d2 = 2.71;
    cout << "\nДо swap: d1 = " << d1 << ", d2 = " << d2 << endl;
    templateSwap(d1, d2);
    cout << "После swap: d1 = " << d1 << ", d2 = " << d2 << endl;

    // Поиск максимума
    cout << "\nМаксимум из 5 и 10: " << getMax(x, y) << endl;
    cout << "Максимум из 3.14 и 2.71: " << getMax(d1, d2) << endl;
    cout << "Максимум из 'A' и 'B': " << getMax('A', 'B') << endl;

    // Работа с массивами разных типов
    int intArr[5] = {1, 2, 3, 4, 5};
    double doubleArr[3] = {1.1, 2.2, 3.3};
    char str[10] = "Hello";

    printArray(intArr);
    printArray(doubleArr);
    printArray(str); // Используется специализация для char[10]
}

```



```
    return 0;  
}
```

Практические задания для закрепления

Задание 1: Калькулятор с разными типами параметров

Создайте калькулятор, который использует:

- Параметры по значению для простых операций
- Параметры по ссылке для накопления результата
- Константные параметры для безопасных вычислений

Задание 2: Система управления студентами

Создайте функции для:

- Добавления студента (параметры по значению/ссылке)
- Поиска студента (константные параметры)
- Обновления информации (параметры-указатели)
- Сортировки студентов (ссылки на массивы)

Задание 3: Матричные операции

Реализуйте операции с матрицами:

- Сложение/умножение (константные ссылки на матрицы)
- Транспонирование (параметр-указатель на матрицу)
- Нахождение определителя (рекурсия + разные типы параметров)

Задание 4: Умный массив

Создайте класс/структуру "умный массив" с методами, использующими:

- Константные методы для чтения
- Ссылки для модификации
- Указатели для работы с памятью
- Шаблоны для разных типов данных

Золотые правила выбора типа параметра

1. Используйте параметры по значению для:

- Простых типов (int, double, char, bool)
- Когда нужна безопасность (защита от изменений)
- Когда размер объекта мал

2. Используйте константные ссылки для:

- Больших объектов (структуры, классы, массивы)
- Когда нужно только чтение данных
- Для эффективности (избегание копирования)

3. Используйте обычные ссылки для:

- Когда нужно изменить оригинальный объект
- Для возврата нескольких значений
- Когда копирование дорого или невозможно

4. Используйте указатели для:

- Работы с динамической памятью
- Когда параметр может отсутствовать (nullptr)
- Для массивов (хотя лучше использовать ссылки на массивы)

5. Всегда используйте const:

- Когда функция не должна изменять параметр
- Для параметров, передаваемых по ссылке или указателю
- Для документирования намерений

Эти примеры и задачи покрывают практически все возможные варианты синтаксиса параметров функций в C++. Практикуйтесь с каждым типом, чтобы глубоко понять различия и правильно выбирать нужный тип параметра для каждой ситуации!

Полное руководство по рекурсии в C++



Что такое рекурсия?

Рекурсия - это когда функция вызывает саму себя для решения задачи. Представьте матрешку: чтобы открыть большую, нужно открыть меньшую внутри нее, и так до самой

маленькой.

Почему рекурсия важна?

- Упрощает код для определенных задач
 - Позволяет решать задачи, которые естественно рекурсивны по своей природе
 - Основа для многих алгоритмов (поиск, сортировка, обход деревьев)
-

Часть 1: Основы рекурсии

Правило №1: Всегда должно быть условие выхода!

Без условия выхода рекурсия будет бесконечной, как бег по кругу.

```
#include <iostream>
using namespace std;

// ПРАВИЛЬНЫЙ пример: есть условие выхода
void countdown(int n) {
    if (n <= 0) { // ← УСЛОВИЕ ВЫХОДА!
        cout << "Пуск!" << endl;
        return; // выходим из рекурсии
    }

    cout << n << "... ";
    countdown(n - 1); // ← РЕКУРСИВНЫЙ ВЫЗОВ
}

// НЕПРАВИЛЬНЫЙ пример: нет условия выхода
void infiniteRecursion(int n) {
    cout << n << " ";
    infiniteRecursion(n + 1); // Бесконечный цикл!
    // Программа "упадет" со Stack Overflow
}

int main() {
    cout << "Пример правильной рекурсии:" << endl;
    countdown(5);

    // Не запускайте эту строку! Она приведет к падению программы
    // infiniteRecursion(1);

    return 0;
}
```

```

/*
Вывод:
Пример правильной рекурсии:
5... 4... 3... 2... 1... Пуск!
*/

```

Визуализация рекурсии countdown(3):

```

countdown(3)
├── Печатает: "3... "
├── Вызывает: countdown(2)
│   ├── Печатает: "2... "
│   ├── Вызывает: countdown(1)
│   │   ├── Печатает: "1... "
│   │   ├── Вызывает: countdown(0)
│   │   │   ├── Печатает: "Пуск!"
│   │   │   └── Возврат (базовый случай)
│   │   └── Возврат
│   └── Возврат
└── Возврат

```

Часть 2: Классические примеры рекурсии

Пример 1: Факториал (самый простой пример)

```

#include <iostream>
using namespace std;

// Факториал: n! = n * (n-1) * (n-2) * ... * 1
// Например: 5! = 5 * 4 * 3 * 2 * 1 = 120
// 0! = 1 (по определению)

long long factorial(int n) {
    // 1. БАЗОВЫЙ СЛУЧАЙ (условие выхода)
    if (n <= 1) {
        return 1; // 0! = 1, 1! = 1
    }
}

```

```

    // 2. РЕКУРСИВНЫЙ СЛУЧАЙ
    //  $n! = n * (n-1)!$ 
    return n * factorial(n - 1);
}

// Визуализация для factorial(4):
// factorial(4) = 4 * factorial(3)
// factorial(3) = 3 * factorial(2)
// factorial(2) = 2 * factorial(1)
// factorial(1) = 1 ← БАЗОВЫЙ СЛУЧАЙ
// Затем поднимаемся обратно:
// factorial(2) = 2 * 1 = 2
// factorial(3) = 3 * 2 = 6
// factorial(4) = 4 * 6 = 24

int main() {
    for (int i = 0; i <= 10; i++) {
        cout << i << "! = " << factorial(i) << endl;
    }

    // Важно: факториал растет очень быстро!
    cout << "\n15! = " << factorial(15) << endl;
    cout << "20! = " << factorial(20) << endl;
    // Для больших чисел используйте long long

    return 0;
}

```

Пример 2: Числа Фибоначчи

```

#include <iostream>
using namespace std;

// Числа Фибоначчи:
//  $F(0) = 0$ ,  $F(1) = 1$ 
//  $F(n) = F(n-1) + F(n-2)$ 
// Последовательность: 0, 1, 1, 2, 3, 5, 8, 13, 21, ...

int fibonacci(int n) {
    // 1. БАЗОВЫЕ СЛУЧАИ
    if (n == 0) return 0;
    if (n == 1) return 1;

    // 2. РЕКУРСИВНЫЙ СЛУЧАЙ
    return fibonacci(n - 1) + fibonacci(n - 2);
}

// Проблема: НЕЭФФЕКТИВНО!

```

```
// fibonacci(5) вызывает fibonacci(4) и fibonacci(3)
// fibonacci(4) вызывает fibonacci(3) и fibonacci(2)
// fibonacci(3) вызывается ДВАЖДЫ! ← лишняя работа
// Для больших n будет очень медленно

// Визуализация дерева вызовов для fibonacci(4):
//          fib(4)
//        /      \
//      fib(3)    fib(2)
//     /  \      /  \
//  fib(2) fib(1) fib(1) fib(0)
//   /  \
// fib(1) fib(0)

int main() {
    cout << "Числа Фибоначчи (первые 15):" << endl;
    for (int i = 0; i < 15; i++) {
        cout << "F(" << i << ") = " << fibonacci(i) << endl;
    }

    // Измеряем время (для демонстрации неэффективности)
    cout << "\nВычисляем F(40) - это будет медленно..." << endl;
    cout << "F(40) = " << fibonacci(40) << endl;

    return 0;
}
```

Пример 3: Сумма цифр числа

```
#include <iostream>
using namespace std;

// Функция для вычисления суммы цифр числа
// Например: sumDigits(123) = 1 + 2 + 3 = 6
int sumDigits(int n) {
    // 1. БАЗОВЫЙ СЛУЧАЙ: если число меньше 10, это одна цифра
    if (n < 10) {
        return n;
    }

    // 2. РЕКУРСИВНЫЙ СЛУЧАЙ:
    // Последняя цифра: n % 10
    // Остаток числа: n / 10
    // Сумма = последняя цифра + сумма цифр остатка
    return (n % 10) + sumDigits(n / 10);
}

// Визуализация для sumDigits(1234):
```

```

// sumDigits(1234) = 4 + sumDigits(123)
// sumDigits(123) = 3 + sumDigits(12)
// sumDigits(12) = 2 + sumDigits(1)
// sumDigits(1) = 1 ← БАЗОВЫЙ СЛУЧАЙ
// Затем поднимаемся:
// sumDigits(12) = 2 + 1 = 3
// sumDigits(123) = 3 + 3 = 6
// sumDigits(1234) = 4 + 6 = 10

// Альтернативный вариант с отладочной печатью
int sumDigitsDebug(int n, int depth = 0) {
    // Отступ для визуализации глубины рекурсии
    string indent(depth * 2, ' ');

    cout << indent << "sumDigitsDebug(" << n << ")" << endl;

    if (n < 10) {
        cout << indent << "Базовый случай: возвращаем " << n << endl;
        return n;
    }

    int lastDigit = n % 10;
    int rest = n / 10;

    cout << indent << "Рекурсивный случай: " << lastDigit << " + sumDigit

    int result = lastDigit + sumDigitsDebug(rest, depth + 1);

    cout << indent << "Возвращаем: " << result << endl;
    return result;
}

int main() {
    int numbers[] = {123, 4567, 987654, 1001, 0};

    for (int num : numbers) {
        cout << "Сумма цифр числа " << num << " = " << sumDigits(num) <<
    }

    cout << "\n=== Демонстрация с отладкой ===" << endl;
    cout << "Сумма цифр числа 1234:" << endl;
    sumDigitsDebug(1234);

    return 0;
}

```

Часть 3: Рекурсия с массивами

Пример 4: Поиск максимума в массиве

```
#include <iostream>
using namespace std;

// Рекурсивный поиск максимума в массиве
int findMax(const int arr[], int start, int end) {
    // 1. БАЗОВЫЙ СЛУЧАЙ: если один элемент, он и есть максимум
    if (start == end) {
        return arr[start];
    }

    // 2. РЕКУРСИВНЫЙ СЛУЧАЙ:
    // Делим массив пополам, ищем максимум в каждой половине,
    // затем возвращаем больший из двух

    int mid = (start + end) / 2;
    int maxLeft = findMax(arr, start, mid);
    int maxRight = findMax(arr, mid + 1, end);

    return (maxLeft > maxRight) ? maxLeft : maxRight;
}

// Более простой вариант: линейная рекурсия
int findMaxLinear(const int arr[], int n) {
    // Базовый случай: последний элемент
    if (n == 1) {
        return arr[0];
    }

    // Находим максимум в оставшейся части массива
    int maxInRest = findMaxLinear(arr + 1, n - 1);

    // Сравниваем первый элемент с максимумом остальной части
    return (arr[0] > maxInRest) ? arr[0] : maxInRest;
}

int main() {
    int arr[] = {3, 7, 2, 9, 1, 4, 6, 8, 5};
    int size = 9;

    cout << "Массив: ";
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    cout << "Максимум (деление пополам): " << findMax(arr, 0, size - 1) <
    cout << "Максимум (линейный): " << findMaxLinear(arr, size) << endl;
```



```
    return 0;  
}
```

Пример 5: Разворот массива

```
#include <iostream>  
using namespace std;  
  
// Рекурсивный разворот массива  
void reverseArray(int arr[], int start, int end) {  
    // Базовый случай: если start >= end, массив пуст или имеет один элемент  
    if (start >= end) {  
        return;  
    }  
  
    // Меняем местами первый и последний элементы  
    int temp = arr[start];  
    arr[start] = arr[end];  
    arr[end] = temp;  
  
    // Рекурсивно разворачиваем оставшуюся часть  
    reverseArray(arr, start + 1, end - 1);  
}  
  
void printArray(const int arr[], int size) {  
    for (int i = 0; i < size; i++) {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
}  
  
int main() {  
    int arr1[] = {1, 2, 3, 4, 5};  
    int size1 = 5;  
  
    int arr2[] = {10, 20, 30, 40, 50, 60};  
    int size2 = 6;  
  
    cout << "Массив 1 до разворота: ";  
    printArray(arr1, size1);  
    reverseArray(arr1, 0, size1 - 1);  
    cout << "Массив 1 после разворота: ";  
    printArray(arr1, size1);  
  
    cout << "\nМассив 2 до разворота: ";  
    printArray(arr2, size2);  
    reverseArray(arr2, 0, size2 - 1);  
    cout << "Массив 2 после разворота: ";
```

```
    printArray(arr2, size2);

    return 0;
}
```

Часть 4: Хвостовая рекурсия (Tail Recursion)

Хвостовая рекурсия - это когда рекурсивный вызов является ПОСЛЕДНЕЙ операцией в функции. Компиляторы могут оптимизировать хвостовую рекурсию.

```
#include <iostream>
using namespace std;

// Обычная рекурсия для факториала
long long factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1); // НЕ хвостовая: умножение после вызова
}

// ХВОСТОВАЯ рекурсия для факториала
// Используем аккумулятор (accumulator) для хранения результата
long long factorialTail(int n, long long acc = 1) {
    if (n <= 1) return acc;
    return factorialTail(n - 1, n * acc); // Хвостовая: вызов - последний
}

// Обычная рекурсия для суммы цифр
int sumDigits(int n) {
    if (n < 10) return n;
    return (n % 10) + sumDigits(n / 10); // НЕ хвостовая: сложение после
}

// ХВОСТОВАЯ рекурсия для суммы цифр
int sumDigitsTail(int n, int sum = 0) {
    if (n == 0) return sum;
    return sumDigitsTail(n / 10, sum + (n % 10)); // Хвостовая
}

int main() {
    cout << "Обычный факториал 5: " << factorial(5) << endl;
    cout << "Хвостовой факториал 5: " << factorialTail(5) << endl;

    cout << "\nОбычная сумма цифр 12345: " << sumDigits(12345) << endl;
    cout << "Хвостовая сумма цифр 12345: " << sumDigitsTail(12345) << endl;

    // Проверка больших чисел
}
```

```
    cout << "\nХвостовой факториал 20: " << factorialTail(20) << endl;

    return 0;
}
```

Часть 5: Рекурсивный поиск (Backtracking)

```
#include <iostream>
#include <vector>
using namespace std;

// Задача: найти все перестановки массива
void generatePermutations(vector<int>& arr, int start, vector<vector<int>>
    // Базовый случай: если дошли до конца массива
    if (start == arr.size() - 1) {
        result.push_back(arr); // добавляем текущую перестановку
        return;
    }

    // Рекурсивный случай: генерируем перестановки
    for (int i = start; i < arr.size(); i++) {
        // Меняем местами текущий элемент с элементом на позиции start
        swap(arr[start], arr[i]);

        // Генерируем перестановки для остальной части массива
        generatePermutations(arr, start + 1, result);

        // ВОЗВРАТ (Backtracking): отменяем изменение
        swap(arr[start], arr[i]);
    }
}

// Задача: задача о рюкзаке (упрощенная версия)
int knapsack(int capacity, const vector<int>& weights, int index) {
    // Базовый случай: нет места или нет предметов
    if (capacity <= 0 || index >= weights.size()) {
        return 0;
    }

    // Если текущий предмет не помещается, пропускаем его
    if (weights[index] > capacity) {
        return knapsack(capacity, weights, index + 1);
    }

    // Два варианта:
    // 1. Берем текущий предмет
```

```

int take = weights[index] + knapsack(capacity - weights[index], weight

// 2. Не берем текущий предмет
int skip = knapsack(capacity, weights, index + 1);

// Возвращаем лучший вариант
return max(take, skip);
}

int main() {
    // Пример 1: Перестановки
    cout << "=== Перестановки чисел ===" << endl;
    vector<int> numbers = {1, 2, 3};
    vector<vector<int>> permutations;

    generatePermutations(numbers, 0, permutations);

    cout << "Все перестановки [1, 2, 3]:" << endl;
    for (const auto& perm : permutations) {
        for (int num : perm) {
            cout << num << " ";
        }
        cout << endl;
    }

    // Пример 2: Задача о рюкзаке
    cout << "\n=== Задача о рюкзаке ===" << endl;
    vector<int> weights = {2, 3, 4, 5};
    int capacity = 8;

    int maxWeight = knapsack(capacity, weights, 0);
    cout << "Максимальный вес, который можно уложить в рюкзак емкостью "
        << capacity << ": " << maxWeight << endl;

    return 0;
}

```

Часть 6: Полезные инструменты для отладки рекурсии

Инструмент 1: Счетчик глубины рекурсии

```

#include <iostream>
using namespace std;

// Рекурсивная функция с отслеживанием глубины
void recursiveFunction(int n, int depth = 0) {

```

```
// Создаем отступ для визуализации
string indent(depth * 2, ' ');

cout << indent << "Глубина " << depth << ": вызов с n = " << n << endl;

// Базовый случай
if (n <= 0) {
    cout << indent << "Базовый случай достигнут!" << endl;
    return;
}

// Рекурсивный вызов
recursiveFunction(n - 1, depth + 1);

cout << indent << "Возврат с глубины " << depth << endl;
}

// Инструмент 2: Визуализация стека вызовов
void visualizeStack(int n, const string& operation) {
    static int callNumber = 0;
    static vector<string> callStack;

    callNumber++;
    string callInfo = "Вызов #" + to_string(callNumber) +
        ": " + operation + "(" + to_string(n) + ")";
    callStack.push_back(callInfo);

    cout << "\nТекущий стек вызовов:" << endl;
    cout << string(30, '-') << endl;
    for (int i = callStack.size() - 1; i >= 0; i--) {
        cout << callStack[i] << endl;
    }
    cout << string(30, '-') << endl;

    // Симуляция работы функции
    if (n > 0) {
        visualizeStack(n - 1, "рекурсия");
    }

    callStack.pop_back();
}

int main() {
    cout << "=== Отслеживание глубины рекурсии ===" << endl;
    recursiveFunction(3);

    cout << "\n=== Визуализация стека вызовов ===" << endl;
    visualizeStack(3, "начало");
}
```

```
    return 0;  
}
```

Часть 7: Практические задания с решениями

Задание 1: Проверка палиндрома

```
#include <iostream>  
#include <string>  
using namespace std;  
  
bool isPalindrome(const string& str, int start, int end) {  
    // Базовый случай: если один символ или меньше  
    if (start >= end) {  
        return true;  
    }  
  
    // Если символы не совпадают, не палиндром  
    if (str[start] != str[end]) {  
        return false;  
    }  
  
    // Рекурсивно проверяем оставшуюся часть  
    return isPalindrome(str, start + 1, end - 1);  
}  
  
int main() {  
    string test1 = "radar";  
    string test2 = "hello";  
    string test3 = "a";  
    string test4 = "racecar";  
  
    cout << "Слово \"" << test1 << "\" палиндром? "  
        << (isPalindrome(test1, 0, test1.length() - 1) ? "Да" : "Нет") <  
  
    cout << "Слово \"" << test2 << "\" палиндром? "  
        << (isPalindrome(test2, 0, test2.length() - 1) ? "Да" : "Нет") <  
  
    cout << "Слово \"" << test3 << "\" палиндром? "  
        << (isPalindrome(test3, 0, test3.length() - 1) ? "Да" : "Нет") <  
  
    cout << "Слово \"" << test4 << "\" палиндром? "  
        << (isPalindrome(test4, 0, test4.length() - 1) ? "Да" : "Нет") <
```

```
    return 0;  
}
```

Задание 2: Быстрое возведение в степень

```
#include <iostream>  
using namespace std;  
  
// Эффективное рекурсивное возведение в степень  
// Используем свойство:  $a^n = (a^{n/2})^2$   
double power(double base, int exponent) {  
    // Базовый случай  
    if (exponent == 0) {  
        return 1;  
    }  
  
    if (exponent == 1) {  
        return base;  
    }  
  
    // Если степень отрицательная  
    if (exponent < 0) {  
        return 1 / power(base, -exponent);  
    }  
  
    // Если степень четная  
    if (exponent % 2 == 0) {  
        double halfPower = power(base, exponent / 2);  
        return halfPower * halfPower;  
    }  
  
    // Если степень нечетная  
    return base * power(base, exponent - 1);  
}  
  
int main() {  
    cout << "2^10 = " << power(2, 10) << endl;  
    cout << "3^5 = " << power(3, 5) << endl;  
    cout << "5^0 = " << power(5, 0) << endl;  
    cout << "2^(-3) = " << power(2, -3) << endl;  
    cout << "1.5^4 = " << power(1.5, 4) << endl;  
  
    return 0;  
}
```

Задание 3: Ханойские башни

```
#include <iostream>
using namespace std;

void hanoi(int n, char from, char to, char aux) {
    if (n == 1) {
        cout << "Переместить диск 1 с " << from << " на " << to << endl;
        return;
    }

    // Переместить n-1 дисков с from на aux, используя to как вспомогатель
    hanoi(n - 1, from, aux, to);

    // Переместить оставшийся диск с from на to
    cout << "Переместить диск " << n << " с " << from << " на " << to << endl;

    // Переместить n-1 дисков с aux на to, используя from как вспомогатель
    hanoi(n - 1, aux, to, from);
}

int main() {
    int disks = 3;
    cout << "Решение Ханойской башни для " << disks << " дисков:" << endl;
    hanoi(disks, 'A', 'C', 'B');

    return 0;
}
```

Часть 8: Советы и предупреждения

✓ Советы по написанию рекурсии:

1. Всегда сначала пишите базовый случай
2. Убедитесь, что рекурсия движется к базовому случаю
3. Используйте отладку с печатью для понимания потока
4. Рассмотрите хвостовую рекурсию для оптимизации
5. Проверяйте глубину рекурсии (может быть **Stack Overflow**)

⚠ Опасности рекурсии:

1. **Stack Overflow** - слишком глубокая рекурсия
2. **Повторные вычисления** - как в наивном Фибоначчи
3. **Неэффективность** - итерация иногда лучше

4. Сложность отладки - сложнее понять поток выполнения

Когда использовать рекурсию:

-  **Использовать:** задачи на деревья, графы, комбинаторика, divide-and-conquer
-  **Не использовать:** простые циклы, где итерация понятнее

Пример Stack Overflow:

```
#include <iostream>
using namespace std;

void deepRecursion(int n) {
    // Выделяем большой массив на стеке (опасно!)
    int bigArray[10000];

    cout << "Глубина: " << n << endl;

    if (n > 10000) return; // Попытка предотвратить...

    deepRecursion(n + 1);    // ...но стек все равно переполнится
}

int main() {
    try {
        deepRecursion(0);
    } catch (...) {
        cout << "Stack Overflow!" << endl;
    }

    return 0;
}
```

Часть 9: Контрольные вопросы для самопроверки

1. Что такое базовый случай в рекурсии?
 2. Почему важны условия выхода из рекурсии?
 3. Чем хвостовая рекурсия отличается от обычной?
 4. Как рекурсия использует стек вызовов?
 5. В чем недостатки наивной реализации чисел Фибоначчи через рекурсию?
 6. Как можно оптимизировать рекурсивные вычисления?
 7. Когда лучше использовать итерацию вместо рекурсии?
-

Часть 10: Дополнительные задачи для практики

Уровень 1 (начальный):

1. Напишите рекурсивную функцию для вычисления суммы чисел от 1 до N
2. Напишите рекурсивную функцию для подсчета количества цифр в числе
3. Напишите рекурсивную функцию для печати чисел от N до 1

Уровень 2 (средний):

1. Реализуйте бинарный поиск рекурсивно
2. Напишите рекурсивную функцию для генерации всех подмножеств множества
3. Реализуйте алгоритм Евклида для НОД рекурсивно

Уровень 3 (продвинутый):

1. Решите задачу "8 ферзей" с помощью рекурсии и backtracking
 2. Реализуйте рекурсивный алгоритм сортировки слиянием (merge sort)
 3. Напишите рекурсивный парсер для простых арифметических выражений
-

Заключение

Рекурсия - мощный инструмент, который при правильном использовании делает код элегантнее и проще. Ключевые моменты:

1. **Всегда начинайте с базового случая**
2. **Убедитесь, что каждый рекурсивный вызов приближает к базовому случаю**
3. **Используйте отладку для понимания потока выполнения**
4. **Помните об ограничениях стека**
5. **Практикуйтесь на разных задачах**

Начните с простых примеров (факториал, сумма цифр), затем переходите к более сложным (поиск в массиве, backtracking). Со временем вы научитесь "видеть" рекурсивные решения естественным образом!